Main System

Scalable RAG with Async Queues & Distributed Workers

Technical Architecture Documentation

Repository: sumanta1983/rag-asyncq-dist-worker

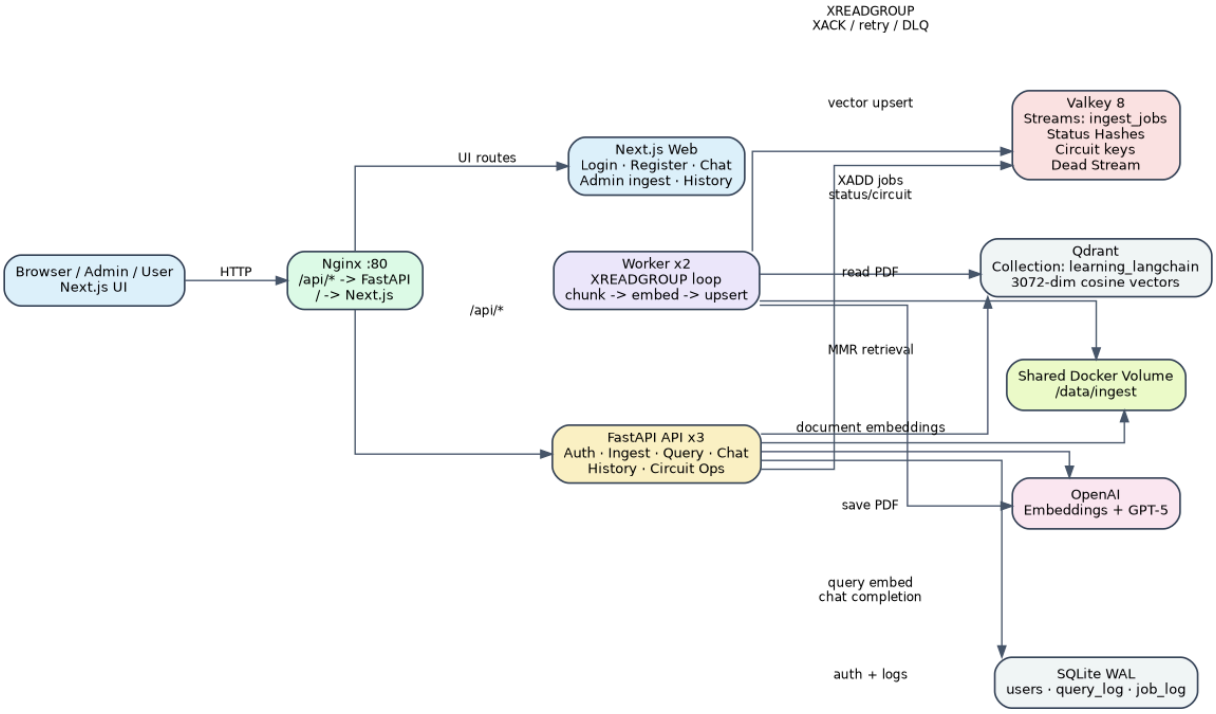

Figure 1. High-level architecture overview.

Prepared from the uploaded ARCHITECTURE.md and the public GitHub repository.

1. Executive Summary

This project converts a synchronous PDF RAG workflow into a distributed, production-oriented RAG platform. The older flow performed load, split, embed, store, retrieve, and answer generation inline. The current system separates fast HTTP work from slow ingestion work by placing Valkey Streams between stateless FastAPI replicas and background workers. Qdrant remains the vector database, OpenAI provides embeddings and GPT-5 responses, and a Next.js UI provides login, chat, admin ingestion, and history screens.

Area	Current design
Frontend	Next.js 15 UI for login, register, chat, admin ingest, and history.
API layer	FastAPI replicas behind Nginx. API owns auth, ingest enqueue, query, chat, history, and ops endpoints.
Queue	Valkey Streams hold ingest jobs and provide at-least-once processing semantics.
Workers	Background consumers load PDF pages, split text, embed chunks, and upsert to Qdrant.
Vector store	Qdrant collection learning_langchain, 3072 dimensions, cosine distance.
Persistence	SQLite stores users, query history, and job history. Valkey stores live job status and circuit breaker state.
Guard rails	JWT auth, admin-only ingestion, OpenAI circuit breaker, retry handling, and dead-letter queue.

Source basis: GitHub repository; uploaded ARCHITECTURE.md; repository README; docker-compose.yml; FastAPI router files; worker files; config files; and nginx.conf.

2. Architecture Overview

The architecture has six practical tiers: browser/UI, reverse proxy, API services, queue/cache, worker services, and persistence/AI dependencies. The most important design decision is that ingestion is asynchronous but query/chat remains synchronous. This keeps the user-facing query path fast while allowing slow PDF embedding to run outside the request/response lifecycle.

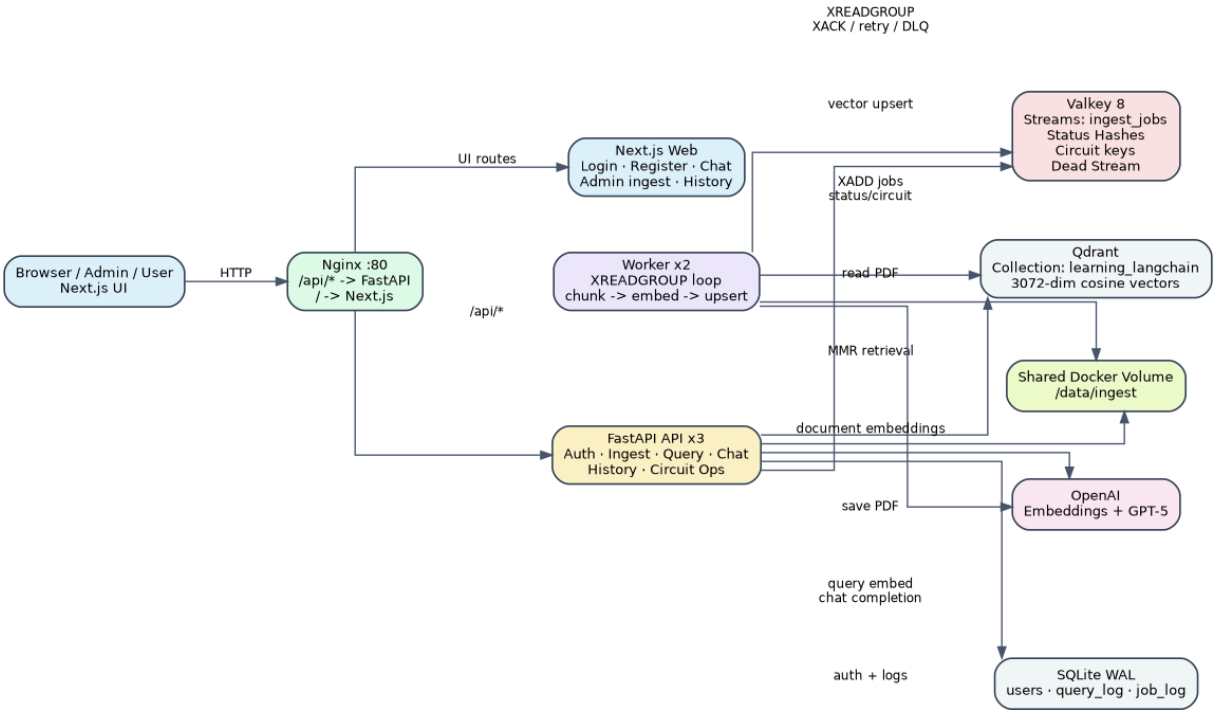

Figure 2. Runtime components and communication paths.

Component	Responsibility	Why it exists
Nginx	Routes /api/* to FastAPI and all other traffic to Next.js.	Single public entry point, reverse proxy, load balancing, 100 MB upload limit, Docker DNS resolution.
FastAPI x3	Auth, ingest enqueue, query, chat, history, circuit controls.	Stateless HTTP tier that can be horizontally scaled.
Next.js web	Login, registration, chat UI, admin ingest UI, history pages.	Gives users and admins a browser-facing application.
Valkey	Streams, status hashes, circuit breaker keys, dead-letter stream.	Buffers ingest jobs and protects the system during OpenAI failures.
Workers x2	Consume ingest jobs, load/split PDFs, call embeddings, write to Qdrant.	Separates long-running CPU/network work from HTTP requests.
Qdrant	Stores embeddings and metadata for retrieval.	Purpose-built vector search database.
SQLite	Users, query_log, job_log.	Simple durable metadata store; WAL mode is acceptable for this small deployment.
OpenAI	text-embedding-3-large for vectors and GPT-5 for final chat answer.	Core AI model dependency.

3. From Sync RAG to Async Distributed RAG

The original synchronous RAG system can be understood as a direct function call chain. The async system keeps the same business logic but redistributes each operation into the correct runtime component.

Original sync responsibility	Distributed responsibility

index_pdf.py reads a local PDF	Admin uploads a PDF through /api/ingest; API saves the file to the shared ingest volume.
PyMuPDFLoader extracts text pages	Worker reads the saved PDF path and loads pages using PyMuPDFLoader.
RecursiveCharacterTextSplitter chunks the document	Worker uses the same splitter style: chunk size 1500, overlap 300, heading-aware separators, start index metadata.
OpenAIEmbeddings embeds all chunks inline	Worker calls OpenAI embeddings outside the HTTP request.
QdrantVectorStore writes vectors inline	Worker uses QdrantVectorStore.add_documents to embed/upsert chunks.
chat_pdf.py runs terminal input and retrieval	/api/query and /api/chat accept HTTP JSON and retrieve with MMR.
LLM answer generated in terminal script	/api/chat builds Baymax context and calls GPT-5.

4. PDF Ingestion Flow

Ingestion is the most important asynchronous workflow. The API performs only cheap work: authenticate the admin, validate the file, save bytes, enqueue metadata, and return a job ID. Workers perform the expensive work later.

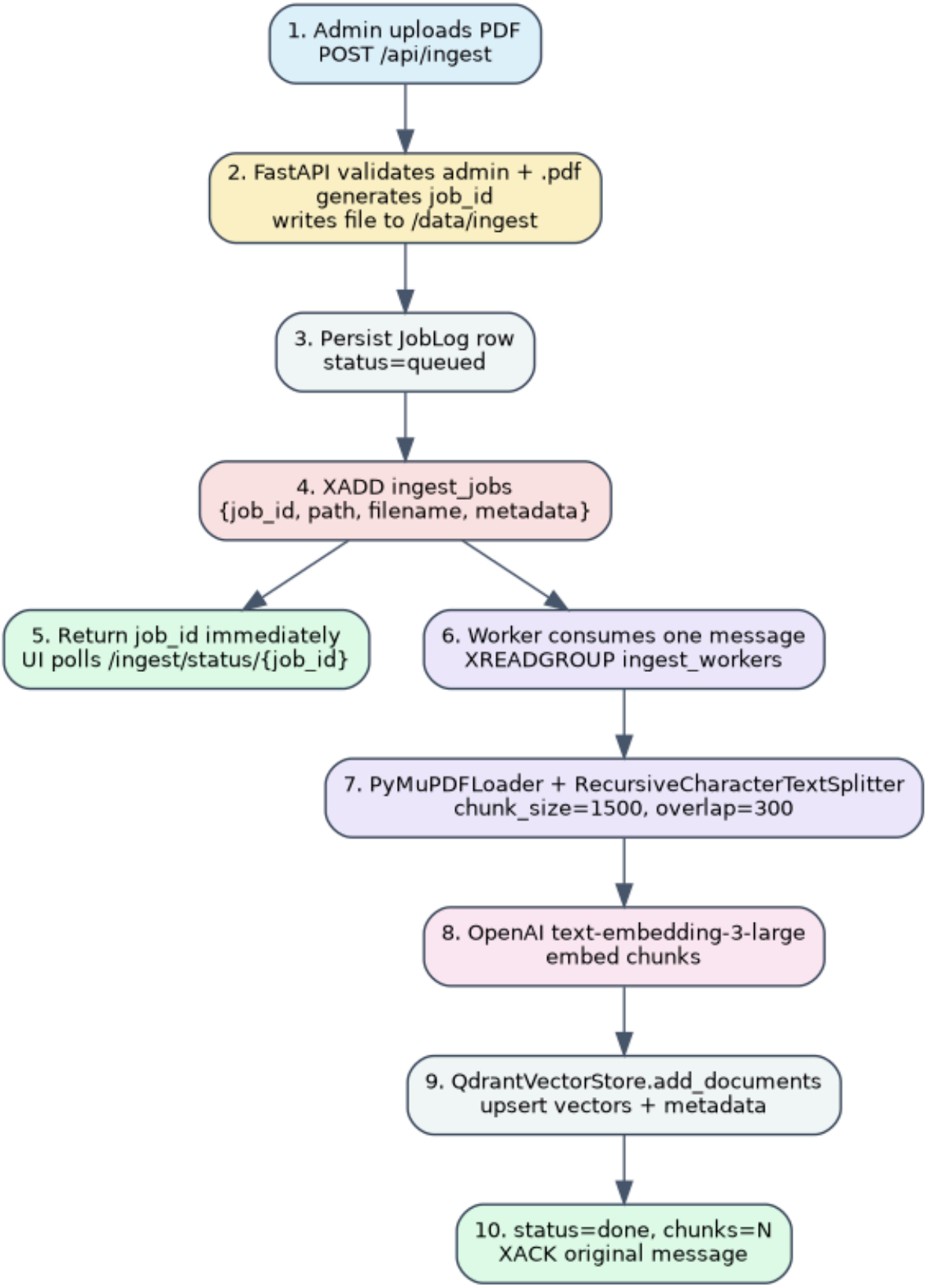
1. Admin uploads PDF
POST /api/ingest
2. FastAPI validates admin + .pdf
generates job_id
writes file to /data/ingest
3. Persist JobLog row
status=queued
4. XADD ingest_jobs
{job_id, path, filename, metadata}
5. Return job_id immediately
UI polls /ingest/status/{job_id}
6. Worker consumes one message
XREADGROUP ingest_workers
7. PyMuPDFLoader + RecursiveCharacterTextSplitter
chunk_size=1500, overlap=300
8. OpenAI text-embedding-3-large
embed chunks
9. QdrantVectorStore.add_documents
upsert vectors + metadata
10. status=done, chunks=N
XACK original message

Figure 3. PDF ingest sequence.

Step	Implementation detail
Admin request	POST /api/ingest accepts multipart file=PDF. The route depends on require_admin, so regular users receive a 403.
Job creation	The API creates uuid4().hex as job_id and writes the PDF to /data/ingest/<job_id>.pdf.
Metadata	Uploaded admin identity is included in the job metadata and later propagated into chunk metadata.
Queue write	Valkey XADD appends job_id, path, filename, and metadata to ingest_jobs.
Live status	Valkey HSET stores job:<job_id> status=queued, stream_id, filename, and upload information.
Durable job log	SQLite JobLog records job_id, user_id, filename, and stream_id.
Worker processing	Worker reads the stream through the ingest_workers consumer group, chunks and embeds the PDF, writes to Qdrant, and acknowledges the message.

```
POST /api/ingest
  -> validate admin and .pdf
  -> save /data/ingest/<job_id>.pdf
  -> XADD ingest_jobs {job_id, path, filename, metadata}
  -> HSET job:<job_id> status=queued
  -> INSERT job_log
  -> return {job_id, stream_id, status}
```

5. Query and Chat Flow

Query and chat are intentionally synchronous because the user is waiting for the answer and retrieval is normally fast. The current implementation uses MMR retrieval for diversity and uses the Baymax system prompt for grounded answers in /chat.

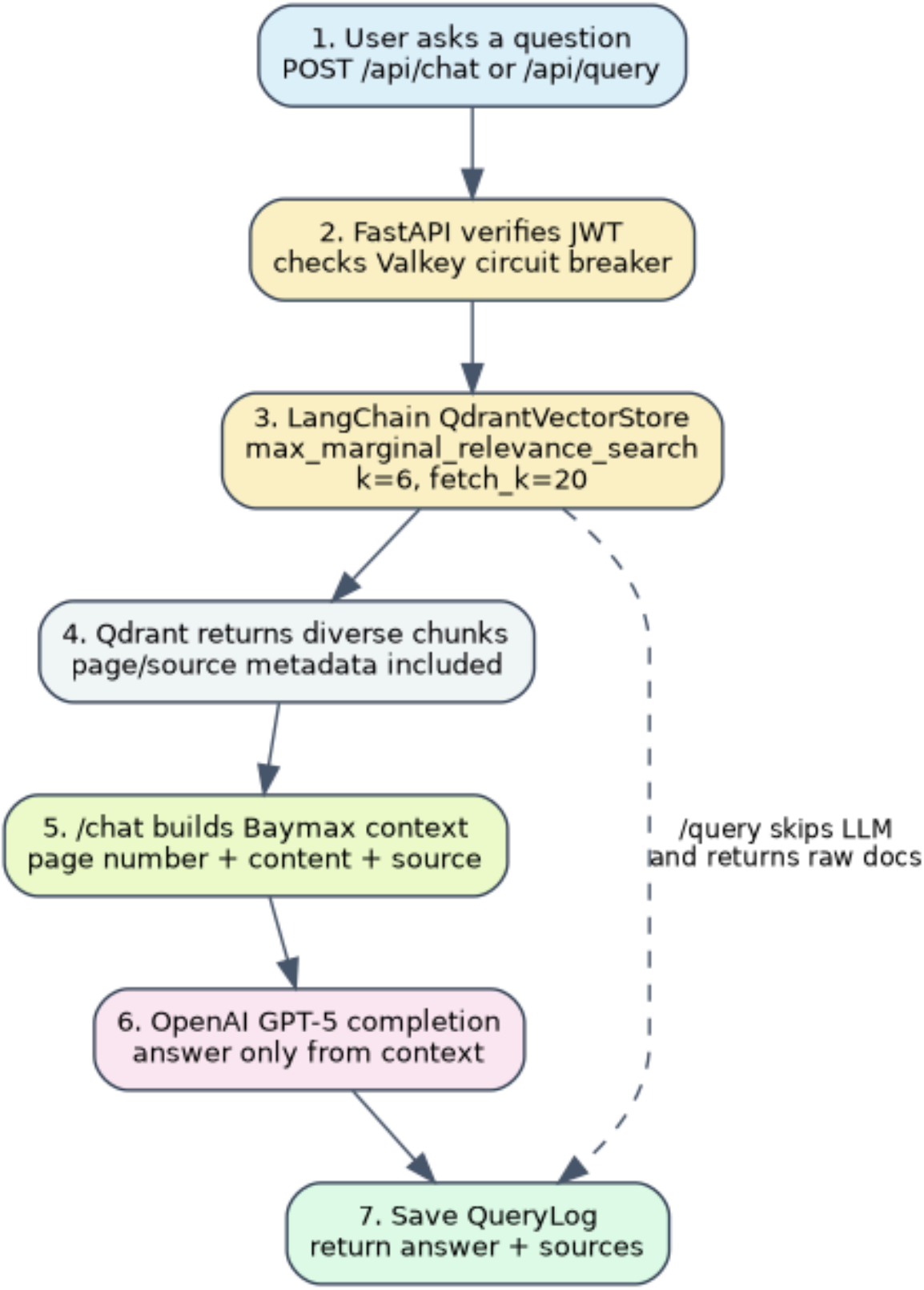
1. User asks a question
POST /api/chat or /api/query
2. FastAPI verifies JWT
checks Valkey circuit breaker
3. LangChain QdrantVectorStore
max_marginal_relevance_search
k=6, fetch_k=20
4. Qdrant returns diverse chunks
page/source metadata included
5. /chat builds Baymax context
page number + content + source
6. OpenAI GPT-5 completion
answer only from context
7. Save QueryLog
return answer + sources
/query skips LLM
and returns raw docs

Figure 4. Query and chat sequence.

Endpoint	Purpose	Output
/api/query	Authenticated retrieval-only endpoint using MMR.	Returns retrieved documents with page_content and metadata.
/api/chat	Authenticated RAG answer endpoint. Retrieves chunks, builds Baymax context, calls GPT-5, logs the answer.	Returns query, model, answer, and sources.
/api/history/queries	Authenticated user history.	Returns a user's own past queries.

6. Code Walkthrough

6.1 docker-compose.yml

The compose file defines the Docker bridge network rag-net and named volumes for Qdrant, Valkey, ingest files, and SQLite. It builds api, worker, and web images, and runs Qdrant, Valkey, and Nginx from official images.

Service	Important configuration
qdrant	qdrant/qdrant:v1.12.4, ports 6333/6334, persistent qdrant_data volume.
valkey	valkey/valkey:8.0-alpine with appendonly enabled, persistent valkey_data volume.
api	QDRANT_URL=http://qdrant:6333, VALKEY_URL=redis://valkey:6379/0, QDRANT_COLLECTION=learning_langchain, CHAT_MODEL=gpt-5, DB_URL=sqlite:////data/app/app.db, replicas=3.
worker	Same Qdrant/Valkey/collection/embed settings; CONSUMER_GROUP=ingest_workers; replicas=2.
web	Next.js dev server; NEXT_PUBLIC_API_BASE=/api; INTERNAL_API_BASE=http://api:8000.
nginx	Listens on port 80 and routes traffic to API or web.

6.2 FastAPI startup

On startup, the API bootstraps the database tables, optionally creates or promotes the first admin from environment variables, and ensures the Qdrant collection exists before serving requests.

```
lifespan:
  _bootstrap_db()
    - create SQLite directory and tables
    - create first admin from ADMIN_MOBILE / ADMIN_PASSWORD
  ensure_collection()
    - create Qdrant collection if missing
  yield
```

6.3 Ingest router

The ingest route is admin-only. It performs file validation, writes the PDF to a shared Docker volume, pushes a Valkey stream message, writes a status hash, and persists a SQLite JobLog row.

```
job = {
  "job_id": job_id,
  "path": "/data/ingest/<job_id>.pdf",
```

```
  "filename": file.filename,
  "metadata": json.dumps(extra)
}
msg_id = valkey.xadd("ingest_jobs", job)
```

6.4 Worker loop

The worker process creates the Valkey consumer group if needed, then repeatedly blocks on XREADGROUP. On success it updates job status and XACKs the message. On failure it increments a retry counter and either re-enqueues or dead-letters the job.

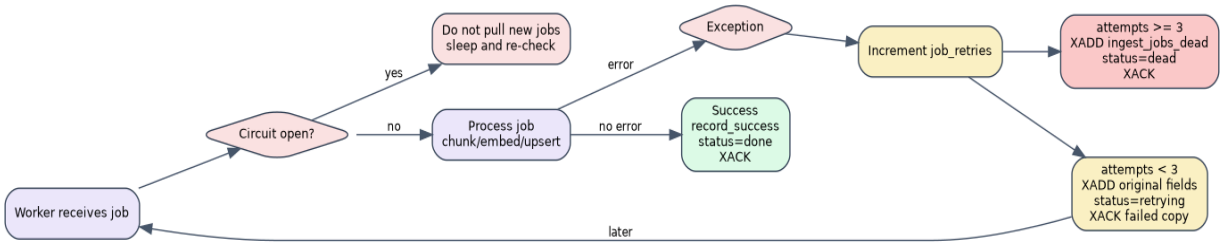

Figure 5. Worker retry, dead-letter, and circuit-breaker flow.

6.5 Chunking and vector storage

The worker uses PyMuPDFLoader for page extraction and RecursiveCharacterTextSplitter for chunking. The splitter configuration mirrors the original rag_system/index_pdf.py behavior: 1500 character chunks, 300 character overlap, Markdown heading-aware separators, and start_index metadata.

```
PyMuPDFLoader(file_path=pdf_path).load()
RecursiveCharacterTextSplitter(
  chunk_size=1500,
  chunk_overlap=300,
  separators=["\n## ", "\n### ", "\n\n", "\n", " ", ""],
  add_start_index=True
)
```

6.6 Query and chat routers

The query router returns raw retrieval results. The chat router wraps the same retrieval with a Baymax prompt and a GPT-5 completion. Both endpoints check the circuit breaker before making OpenAI calls and log successful requests to SQLite.

```
results = get_vector_store().max_marginal_relevance_search(
  query=req.query,
  k=req.k or settings.mmr_k,
  fetch_k=req.fetch_k or settings.mmr_fetch_k
)
```

7. API Reference

Method	Path	Access	Description
POST	/api/auth/register	Public	Create a user account with name, mobile, password, and confirm_password.
POST	/api/auth/login	Public	Login and receive a bearer token.
GET	/api/auth/me	Authenticated	Return the current user.

POST	/api/ingest	Admin	Upload and enqueue a PDF ingest job.
GET	/api/ingest/status/{job_id}	Admin	Read live ingest status from Valkey.
POST	/api/query	Authenticated	Run MMR retrieval and return source documents.
POST	/api/chat	Authenticated	Run MMR retrieval, build Baymax context, call GPT-5, and return answer with sources.
GET	/api/history/queries	Authenticated	Return current user's query history.
GET	/api/history/jobs	Admin	Return ingest job history.
GET	/api/circuit	Admin	View circuit breaker status.
POST	/api/circuit/disable	Admin	Manual OpenAI kill switch.
POST	/api/circuit/enable	Admin	Re-enable OpenAI calls.
GET	/healthz	Public	Health check.

8. Data and Storage Design

Store	Data	Lifecycle
Qdrant	Document chunk vectors, text payload, source/page/start_index/job/user metadata.	Persistent vector corpus used by /query and /chat.
Valkey stream ingest_jobs	Queued PDF ingestion messages.	Messages are acknowledged after successful worker processing or after retry/dead-letter handling.
Valkey hash job:<job_id>	Live status: queued, processing, retrying, done, dead.	Expires after status_ttl, default 86400 seconds.
Valkey stream ingest_jobs_dead	Failed jobs after max deliveries.	Operational inspection and manual recovery.
SQLite users	Mobile, name, password hash, role.	Persistent auth data.
SQLite query_log	User query, endpoint, answer, model.	User history and audit trail.
SQLite job_log	Admin upload jobs.	Admin job history.
Shared ingest volume	Uploaded PDF files by job_id.	Read by workers after API upload.

9. Security, Reliability, and Scaling

Concern	Current handling	Recommendation
Authentication	JWT-based login/register; bearer token required for protected endpoints.	Keep JWT_SECRET strong and rotate when needed. Add refresh tokens for long-lived sessions.
Authorization	/ingest, job history, and circuit ops are admin-only.	Continue enforcing role checks at API dependency level, not just UI level.
Password storage	bcrypt password hashing.	Add password reset and rate limiting before production exposure.
OpenAI failures	Circuit breaker opens after configured failure threshold and cools down for 60 seconds.	Expose clear admin dashboard status and alerts.
Job failures	Retry count with max_deliveries=3 and dead-letter stream.	Add PEL claim loop for workers that die after receiving but before handling messages.
Scaling API	docker compose up -d --scale api=5.	Stateless design supports this; watch SQLite write contention.
Scaling workers	docker compose up -d --scale worker=4.	Add worker concurrency limits so OpenAI quota is not exceeded.
Database	SQLite WAL on shared volume.	Move to Postgres when write contention or multi-host deployment appears.

10. Local Deployment Runbook

```
cp .env.example .env
# Set at minimum:
#   OPENAI_API_KEY
#   JWT_SECRET
```

```
#   ADMIN_MOBILE
#   ADMIN_PASSWORD

docker compose up --build -d
docker compose ps
# Open http://localhost and login with the first admin credentials
```

Task	Command / action
Scale API	docker compose up -d --scale api=5
Scale workers	docker compose up -d --scale worker=4
Admin upload via UI	Open /admin/ingest, upload PDF, watch status polling.
CLI upload	Login, extract access_token, call curl -F file=@doc.pdf -H Authorization: Bearer <token> http://localhost/api/ingest.
Health check	GET /healthz through Nginx.

11. Recommended Next Improvements

- Add a PEL claim loop: workers should periodically XAUTOCLAIM/XCLAIM pending messages older than a safe threshold so jobs are not stranded after a hard worker crash.
- Add SSE streaming for /chat and set Nginx proxy_buffering off for that route so long answers appear token-by-token.
- Move SQLite to Postgres when scaling beyond one host or when concurrent writes become visible.
- Add integration tests for ingest, retry, dead-letter, auth, query, and chat flows.
- Add rate limiting for auth and chat endpoints to protect cost and prevent abuse.
- Add document deletion/reindexing support so admins can update a corpus cleanly.
- Add observability: request IDs, structured logs, metrics for queue depth, worker duration, OpenAI errors, and Qdrant latency.
- Add retrieval evaluation: store question-answer test sets and measure answer faithfulness and source recall after changes.

12. Glossary

Term	Meaning
RAG	Retrieval-Augmented Generation: retrieve relevant knowledge first, then generate an answer grounded in that context.
MMR	Maximal Marginal Relevance: retrieval that balances similarity and diversity to reduce repeated chunks.
Valkey Stream	Redis-compatible append-only stream used here as the ingest queue.
Consumer group	Stream feature that distributes messages across workers while tracking pending/unacknowledged messages.
Dead-letter queue	A separate stream for jobs that exceeded retry limits.
Circuit breaker	A guard that stops OpenAI calls temporarily after repeated failures.
Embedding	A numerical vector representation of text used for semantic search.
Qdrant collection	A named vector index/table containing vectors and payload metadata.

13. References

- **Public repository:** https://github.com/sumanta1983/rag-asyncq-dist-worker
- **Uploaded architecture note:** ARCHITECTURE.md provided in the conversation
- **README content reviewed from repository:** Repository README on dev branch

- **Implementation files reviewed:** docker-compose.yml, api/app/main.py, api/app/routers/ingest.py, api/app/routers/query.py, api/app/routers/chat.py, worker/app/main.py, worker/app/pipeline.py, worker/app/chunker.py, api/app/config.py, worker/app/config.py, api/app/deps.py, nginx/nginx.conf

Feature - Ingestion Quality

Ingestion Quality Feature

Design, architecture, and operational guide

Project: rag-asyncq-dist-worker

Version: 1.0 · Audience: engineers, reviewers, ops

Contents

1. Motivation

Before this feature, the worker chunked every uploaded PDF and pushed the resulting text fragments straight to OpenAI for embedding and then to Qdrant. There was no observability into the shape of the data going in: empty pages, scanner artefacts, running headers/footers repeated on every page, and duplicate paragraphs all ended up as paid embeddings and as noisy vectors in the index.

The feature adds three things:

- A cheap, deterministic quality check on every chunk (length, lexical diversity, optional encoding noise) before it reaches OpenAI.
- Per-job aggregate reporting stored in SQLite so quality can be inspected after the fact.
- An admin-only dashboard page in the Next.js UI to browse the report history.

Concrete benefits:

- Cost — bad chunks don't consume embedding tokens (text-embedding-3-large is billed per token).
- Retrieval quality — duplicates inside one PDF do not pollute MMR neighbours.
- Visibility — admins can see immediately why an ingest produced fewer chunks than expected.
- Safe by default — the dedupe and validation are tunable; non-English PDFs are not penalised.

2. System architecture

The system is a five-service Docker compose stack on a single bridge network (rag-net). Stateless API replicas sit behind Nginx; stateful work is partitioned between Valkey (queue + cache), Qdrant (vectors), and a shared SQLite file mounted into both api and worker containers.

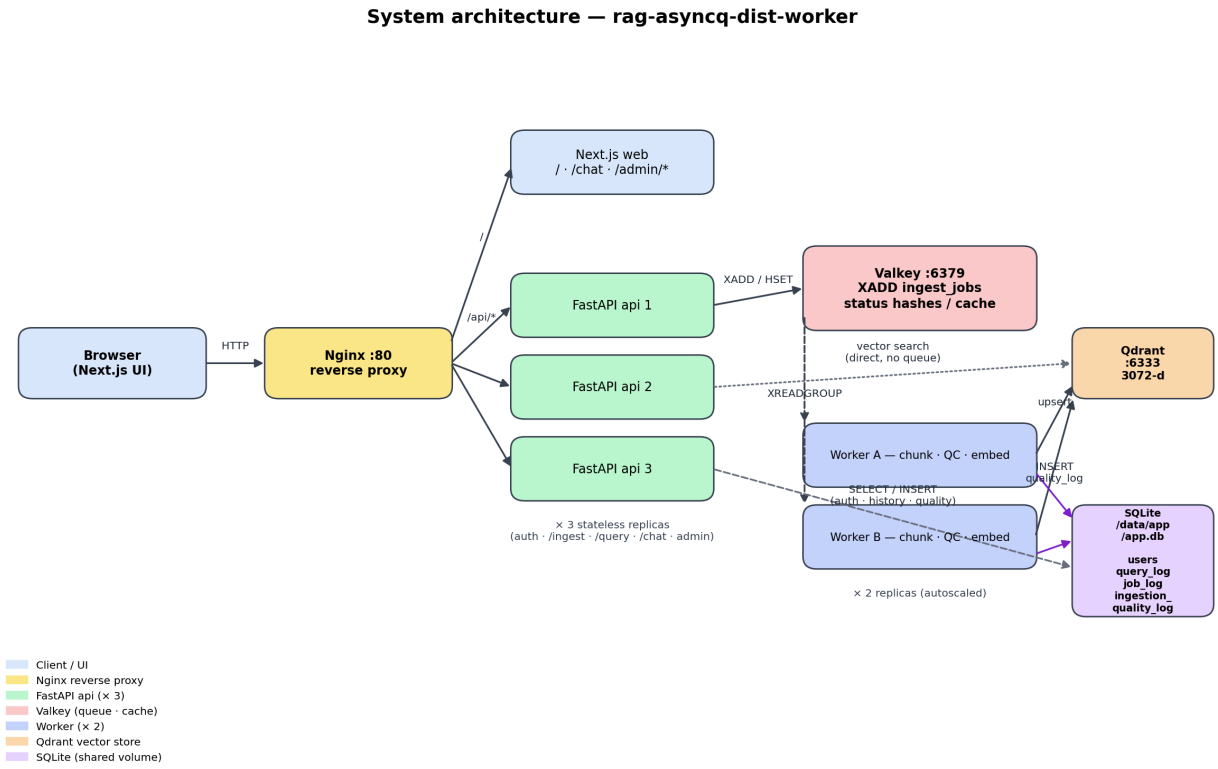

Figure 1 — Container topology. Quality reports flow from worker ⟶ SQLite; API reads back for the admin dashboard.

What runs where

Container	Replicas	Role
Nginx	1	Reverse proxy: '/' ⟶ Next.js, '/api/*' ⟶ FastAPI
Next.js web	1	UI: login, register, chat, admin/ingest, admin/ingestion-quality
FastAPI api	3	Stateless HTTP: auth, ingest enqueue, retrieval, admin endpoints
Valkey	1	Job stream (ingest_jobs) + job-status hashes + circuit-breaker flag
Worker	2	Chunk ⟶ quality check ⟶ embed ⟶ Qdrant upsert + SQLite quality log

Qdrant	1	HNSW vector index, 3072-dim cosine
SQLite (volume)	—	Single file on sqlite_data volume mounted on both api and worker

Key invariant: the SQLite volume sqlite_data is mounted on both api and worker. Workers INSERT quality rows; API SELECTs them for the dashboard. SQLite WAL mode and a 30-second busy timeout are configured in db.py to handle the cross-container writer/reader pattern.

3. End-to-end ingestion + quality flow

When an admin uploads a PDF the work splits in two: a fast synchronous path that enqueues the job and returns a job_id, and an asynchronous worker path that does the heavy lifting.

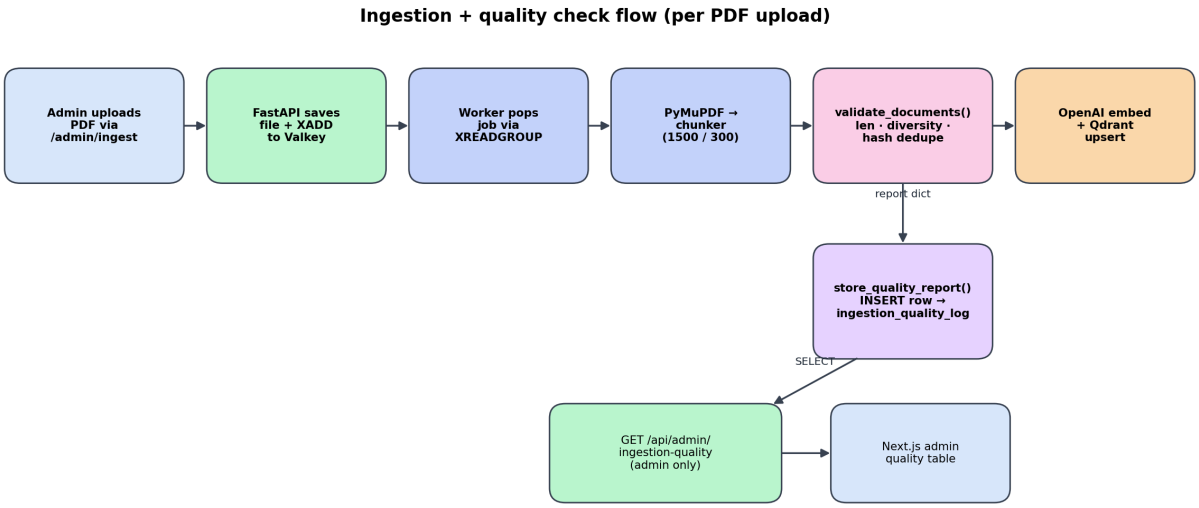

Figure 2 — Per-PDF flow. The quality report is a side-effect of the embed pipeline, written to SQLite while the surviving chunks continue to Qdrant.

Step-by-step

1. Admin POSTs the PDF to /api/ingest with a Bearer JWT. FastAPI streams the file to /data/ingest/<job_id>.pdf on the shared ingest_data volume.
2. FastAPI XADDs an entry to the ingest_jobs Valkey stream containing job_id, path, filename, and any metadata. It returns { job_id, stream_id } in milliseconds.
3. A worker blocked on XREADGROUP picks up exactly one copy of the message (consumer-group semantics guarantee exclusivity).
4. Worker runs PyMuPDFLoader and RecursiveCharacterTextSplitter (1500/300) to produce chunks.
5. Worker calls validate_documents() (worker/app/ingest_checks.py). Each chunk is scored on length, lexical diversity, optional non-ASCII ratio. Identical-hash duplicates are dropped.
6. Worker calls store_quality_report() (worker/app/quality_store.py) which INSERTs one row into ingestion_quality_log with the aggregate counts and JSON-encoded issue summary.
7. Surviving chunks are embedded with text-embedding-3-large and upserted to Qdrant.
8. Worker XACKs the stream message. The next admin GET on /api/admin/ingestion-quality returns the new row.

4. The quality checks themselves

The validator is intentionally deterministic and cheap. It does not call any LLM and runs in O(chunks) time with no I/O. The chosen thresholds are documented inline in worker/app/ingest_checks.py.

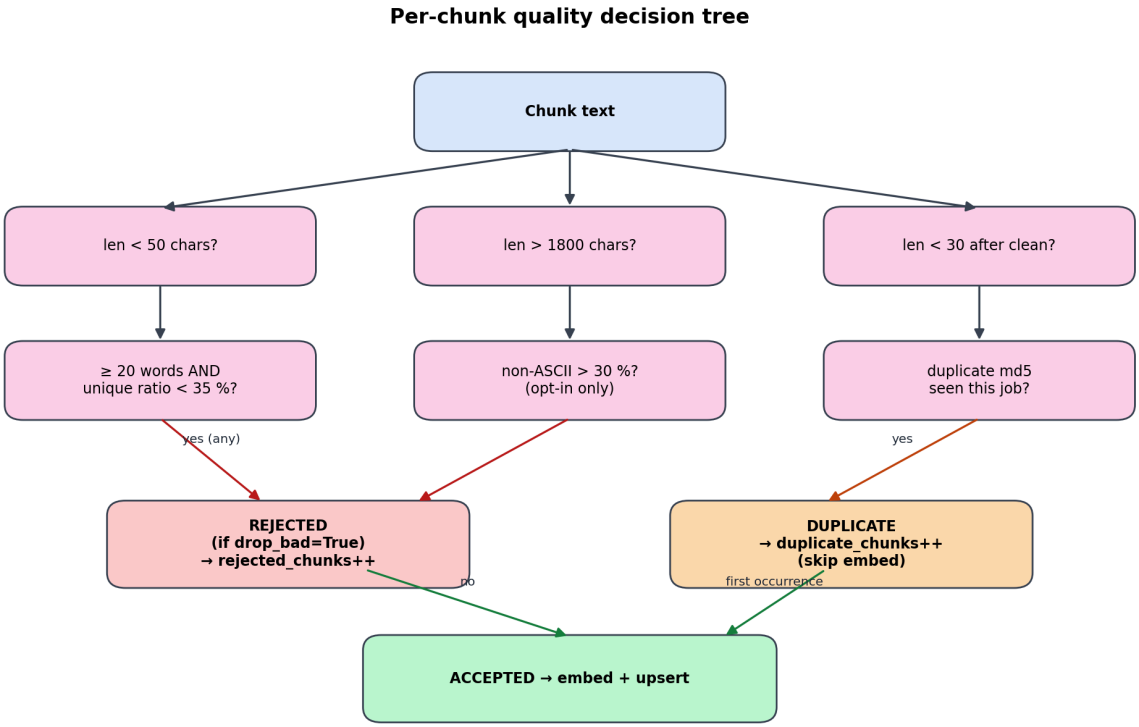

Figure 3 — Decision tree. Length, diversity, and noise checks decide rejection; an MD5 of the normalised chunk text decides duplication.

Per-check rationale

Check	Default threshold	Why this default?
Min length	50 chars	Below this is almost always a heading line or page artefact, not a retrievable answer chunk.
Max length	1800 chars	Equal to chunk_size + chunk_overlap; anything larger means the splitter mis-split. Flag, don't drop blindly.
Effectively empty	30 chars after whitespace collapse	Catches chunks that look long but are mostly whitespace/page numbers.
Lexical diversity	unique-word ratio < 35 % when ≥ 20 words	Detects boilerplate like running headers. Word count gate prevents short legit chunks from being penalised.
Non-ASCII noise	> 30 % non-ASCII	OFF by default. Bengali/Hindi PDFs would be 100 % flagged

		otherwise. Enable for English-only corpora.
Duplicate (md5)	Exact match of normalised text within one job	Removes per-page repeats (running headers, license notices) cheaply.

Why MD5 and not a similarity hash?

Within a single PDF the most common dupes are byte-identical (running headers, table-of-contents repeats, license footers). MD5 of the whitespace-collapsed text catches them with no false positives. Cross-document near-dupes are a different problem and would need a SimHash or embedding-space comparison — out of scope for this version.

5. Data model — SQLite schema

The new ingestion_quality_log table sits alongside the existing users / query_log / job_log tables. It is keyed on job_id rather than a foreign-key relationship so the worker doesn't need to know about the api-side User model.

Figure 4 — SQLite schema. ingestion_quality_log is decoupled by design; join on job_id at read time when an admin wants per-user attribution.

Column reference

Column	Type	Source / meaning
id	INTEGER PK	auto
job_id	TEXT, indexed	Same job_id used by job_log.job_id and Redis status hash
filename	TEXT NULL	Original upload filename (best-effort)
original_chunks	INTEGER	Output of the splitter (before validation)
checked_chunks	INTEGER	Equal to original_chunks today; reserved for future sampling
kept_chunks	INTEGER	Chunks actually upserted to Qdrant
rejected_chunks	INTEGER	Failed at least one quality rule
duplicate_chunks	INTEGER	Hash-deduped within this job
quality_passed	BOOLEAN	rejected == 0 AND duplicates == 0
issues_json	TEXT (JSON)	{ issue → count } summary

sample_issues_json	TEXT (JSON)	First 5 offending chunks with page + preview
created_at	DATETIME, indexed	UTC timestamp of report write

Why no foreign key from quality_log to job_log?

The worker writes ingestion_quality_log. job_log is owned by the API. Adding an FK from a worker-managed table to an API-managed one would couple the two services and create migration-ordering issues. Joining on the string job_id at read time is cheap and keeps each service independently deployable.

6. REST API surface

Two admin-only endpoints (mounted on api/app/routers/admin_quality.py):

```
GET /api/admin/ingestion-quality
    ?limit=50 (1..200)
    &offset=0
    &only_failed=false
→ { items: QualityRow[], limit, offset, only_failed }

GET /api/admin/ingestion-quality/{job_id}
→ { job_id, items: QualityRow[] }

QualityRow = {
  id, job_id, filename,
  original_chunks, checked_chunks,
  kept_chunks, rejected_chunks, duplicate_chunks,
  quality_passed,
  issues: { issue → count },
  sample_issues: [{ page, source, issues, preview }],
  created_at: ISO-8601
}
```

Auth: both endpoints depend on require_admin from api/app/auth/deps.py. A non-admin JWT receives 403; no token receives 401.

7. Admin UI — Next.js page

Route: /admin/ingestion-quality. Same role-guard pattern used by /admin/ingest: anonymous → redirect to /login; non-admin user → redirect to /chat. The page polls on demand (manual refresh button) rather than a poll loop, because new rows only appear when an ingest finishes.

Columns

- Time — locale-formatted from created_at.
- File — filename (or '—' for old jobs with no filename).
- Job ID — truncated for display, full ID available on hover.
- Original / Kept / Rejected / Duplicates — direct numeric counts.
- Status — green 'Passed' or red 'Failed' pill driven by quality_passed.

Controls:

- "Only failed" toggle — sends only_failed=true to the API.
- Refresh — re-fetches the current page.
- Previous / Next — 50 rows per page; Next is disabled when fewer than 50 rows return.

8. Configuration

All toggles live in worker/app/config.py and are env-var overridable:

Env var	Default	Effect
QUALITY_MIN_LEN	50	Minimum chars per chunk before flagging.
QUALITY_DROP_BAD	true	If true, rejected chunks are dropped before embed. If false, they are kept and the flag is recorded in the chunk's Qdrant payload.
QUALITY_DEDUPE	true	Skip duplicate-hash chunks within a job.
QUALITY_ENABLE_ASCII_NOISE_CHECK	false	Enable only for English-only corpora.
DB_URL	sqlite:////data/app/app.db	Shared with the API. Override to use Postgres if scaling out.

Worker max chunk length is dynamic:

```
max_len = settings.chunk_size + settings.chunk_overlap  # 1500 + 300 = 1800
```

This keeps the upper bound in sync with the splitter's mathematical maximum. If chunk_size changes, the validator follows.

9. Operations

Verifying the feature locally

```
docker compose up --build -d
docker compose ps

# Sign in as admin (see .env: ADMIN_MOBILE / ADMIN_PASSWORD),
# then upload a PDF at http://localhost/admin/ingest.
# Open http://localhost/admin/ingestion-quality
# A row should appear within seconds of the ingest finishing.
```

Common issues and how to diagnose

Symptom	Likely cause	Check
/admin/ingestion-quality is empty even after a successful upload	API container can't read the sqlite_data volume	docker compose exec api ls -l /data/app/app.db — must exist and be readable
Worker logs say 'AttributeError: settings has no attribute quality_*'	Worker built before adding quality settings to config.py	docker compose build worker && docker compose up -d worker
Every chunk is reported as 'high non-ascii ratio'	QUALITY_ENABLE_ASCII_NOISE_CHECK is on for a non-English corpus	Set QUALITY_ENABLE_ASCII_NOISE_CHECK=false and restart workers
All chunks are 'duplicate'	PDF is mostly empty pages, or splitter emitted identical chunks	Open the row's sample_issues to see the offending text
docker compose up fails: 'volume sqlite_data not declared'	Top-level volumes block missing the entry	docker-compose.yml volumes: must include sqlite_data:

Scaling notes

- SQLite in WAL mode tolerates 2 workers + 3 api replicas writing/reading the same file via a Docker volume. Beyond ~20 ingests/min sustained, expect lock contention.
- Migrating to Postgres needs only a DB_URL change; the SQLAlchemy models are dialect-agnostic.
- The quality check itself is single-threaded inside a worker and adds ~10 ms per 100 chunks — negligible next to the OpenAI embedding call.

10. Trade-offs and future work

Conscious trade-offs

- Duplicate detection is per-job, not corpus-wide. Catches the common 'every page has the same header' case; misses 'I uploaded the same PDF twice'. Easy to extend by consulting Qdrant before embed.
- quality_passed = (rejected == 0 AND duplicates == 0) treats dedupe as a quality failure. This is debatable — duplicates are informational, not bad data. Discussed as a known sharp edge.
- Non-ASCII check is opt-in to avoid penalising Indic-language documents — explicit choice to honour the project's multilingual scope.
- ingestion_quality_log has no FK to job_log so the worker stays independent of the API's User schema.

Future work

- Per-job drill-down page: clicking a row should expand sample_issues with previews.
- Time-series view: rejection rate over time would surface gradual corpus quality drift.
- Cross-job dedupe: hash-check against an existing payload field in Qdrant before embed.
- Quality-aware re-ingest: a 'reprocess this PDF with lower min_len' admin action.
- Move to Postgres for >20 ingests/min throughput targets.

Appendix A — file-by-file change log

File	Change
worker/app/ingest_checks.py	New. validate_documents + check_chunk.
worker/app/quality_store.py	New. store_quality_report writes one row per job.
worker/app/models.py	New IngestionQualityLog table model.
worker/app/db.py	Shared SQLAlchemy engine + session_scope; WAL pragmas.
worker/app/pipeline.py	Calls validate_documents and store_quality_report; only valid chunks reach Qdrant.
worker/app/config.py	Added QUALITY_* settings.
worker/app/main.py	Calls _bootstrap_db on start so the table exists.
worker/requirements.txt	Added sqlalchemy.
api/app/models.py	Added matching IngestionQualityLog; removed stray xmlrpc import.
api/app/routers/admin_quality.py	New. List + by-job-id endpoints.
api/app/main.py	Registers admin_quality.router.
docker-compose.yml	Declares sqlite_data volume; mounts it on api + worker; sets DB_URL on api.
web/app/admin/ingestion-quality/page.tsx	New. Table UI with pagination, only-failed filter, status pill.
web/components/Nav.tsx	Adds 'Quality (admin)' link.
README.md	Documents the new endpoints, UI route, and env vars.